Contents

UPDC 1.1.0 Complete User, Algorithm, and Engineering Manual **1**
1. Purpose and design principles . . . 1
2. Mathematical model . . . 2
3. Input data, DQ, background, and weights . . . 3
4. Common target WCS and exact detector-pixel integration . . . 4
5. Detector-frame PSFs and full WCS/GWCS polygon transport . . . 4
6. Numerical precision policy . . . 5
7. Initial estimate and FreezeMu . . . 6
8. Main computational optimizations . . . 6

$$ \sum_k P_k^T s_k **7**
9. Cache lifecycle and reset controls . . . 8
10. TOML configuration and recommended execution . . . 8
11. Installation . . . 10
12. Linux and HPC deployment . . . 10
13. Instrument adapters and generalization . . . 11
14. Choosing parallelism . . . 11
15. Cache ABI . . . 12
16. Outputs and diagnostics . . . 12
17. Testing and scientific regression . . . 13
18. GitHub release workflow . . . 14
19. Known boundaries and capabilities that should not be overstated . . . 14
20. Recommended production principles . . . 14
21. Version information . . . 14
22. MIT License and scientific publication citation policy . . . 15

UPDC 1.1.0 Complete User, Algorithm, and Engineering Manual

Release status: GitHub release / MIT License
Numerical core: frozen v17 C implementation
Configuration: stable TOML schema v1
Scientific acceptance: end-to-end validated for the documented JWST/NIRCam workflow; other instruments are supported through adapters and the normalized FITS contract but require independent scientific validation.

---

1. Purpose and design principles

UPDC is a high-performance package for joint reconstruction of multiple astronomical exposures. The implementation is designed around three principles.

1. **Separate the scientific operator from telescope-specific file formats.** The numerical core consumes exposures, weights, PSFs, and geometric mappings. JWST, generic FITS, and future instruments belong in adapters.
2. **Treat detector geometry rigorously.** A detector pixel is not modeled as a center-point sample. Its four mapped corners define a quadrilateral on the common target plane, and the detector response is represented by exact polygon-overlap area fractions.
3. **Performance optimizations must preserve the scientific operator.** The v17 numerical C core is frozen. FFT scheduling, sparse LinkMap compression, threading, caching, and build-system portability are engineering changes only. Any change that alters the reconstruction operator requires a new core freeze and a new scientific validation campaign.

The protected numerical-core files are:

- `updc/c_core/updc_core.c`
- `updc/c_core/overlap.c`
- `updc/c_core/overlap_geometry.h`
- `updc/c_core/psf_projector.c`

Their SHA-256 values are recorded in `updc/core_freeze.json` and verified by tests and `updc doctor`.

---

2. Mathematical model

For exposure `k`, UPDC defines the forward operator

$$H_k = S_k P_k,$$

where

- P_k is convolution by the exposure-specific PSF on the common target grid;
- S_k integrates the convolved target image over mapped detector pixels;
- W_k is the detector-pixel weight map;
- b_k is the scalar background estimated for that exposure;
- p_k is a non-negative pedestal used to keep the modeled mean valid for the signed multiplicative update;
- x is the signed target reconstruction.

The Level-2 science plane is **not** background-subtracted by the Python driver. For a usable detector pixel,

$$y_{\mathrm{eff},k} = SCI_k + p_k,$$

$$\mu_k = H_k x + b_k + p_k.$$

When $\mu_k > 0$,

$$r_k = \frac{y_{\mathrm{eff},k}}{\mu_k}.$$

If a detector pixel ever reaches $\mu_k \le 0$, that pixel enters persistent **FreezeMu** and its ratio is fixed to

$$r_k = 1$$

for all subsequent iterations. This avoids an invalid or unstable division while retaining the pixel's weight contribution to the adjoint scatter.

The multiplicative update is

$$x \leftarrow x \, \frac{\sum_k H_k^T \left(W_k r_k\right)}{\sum_k H_k^T W_k}.$$

The sensitivity denominator

$$D = \sum_k H_k^T W_k$$

is computed once before the iterative loop and reused.

2.1 Why the reconstruction is called signed-x

The target image is allowed to contain both positive and negative values. The initialization is a weighted drizzle-like estimate with the corresponding scalar sky background removed, and the iterative solver does **not** impose positivity clipping on `x`.

This matters for real astronomical backgrounds because sky noise and residual structure can be negative. UPDC therefore should not be interpreted as an ordinary non-negative Richardson-Lucy implementation.

2.2 Pedestal semantics

The pedestal is a constant offset applied to both the observed and modeled means:

$$SCI + p, \qquad Hx + b + p.$$

It is not a noise realization and does not modify WHT. The driver chooses at least the minimum automatic pedestal required to keep all usable `SCI + pedestal` values positive. `poisson_pedestal_sigma` can impose an additional background-RMS-based lower bound.

---

3. Input data, DQ, background, and weights

The normalized exposure contract used by the frozen core expects same-sized two-dimensional FITS exposures containing:

- `SCI`
- `DQ`
- `VAR_POISSON`
- `VAR_RNOISE`
- `VAR_FLAT`

The JWST adapter can use `meta.wcs` / GWCS. The generic FITS adapter uses celestial FITS WCS from the `SCI` header. Input SCI units must be consistent across the exposure set.

3.1 DQ handling

`bad_dq_mask` is a bit mask. A detector pixel is rejected when

```
(DQ & bad_dq_mask) != 0
```

The default JWST mask is instrument-specific. A generic instrument must select a mask consistent with its own DQ semantics.

3.2 Scalar background estimation

Each exposure receives an independent scalar background estimate. The implementation uses asymmetric sigma clipping that rejects positive source contamination more aggressively than negative excursions, producing a robust sky median and RMS estimate.

3.3 Weight modes

`weight_mode = "rnoise-sky"` is the recommended default for the validated JWST workflow:

$$W = \frac{1}{VAR_{RNOISE} + \mathrm{median}_{sky}(VAR_{POISSON})}.$$

This preserves detector/read-noise pixel-to-pixel variation without directly using bright-source-dependent pixelwise Poisson variance.

`weight_mode = "totalvar"` uses

$$W = \frac{1}{VAR_{POISSON} + VAR_{RNOISE} + VAR_{FLAT}}.$$

`weight_mode = "mask"` assigns unit weight to otherwise usable pixels.

After loading, positive weights are normalized by one global median factor for numerical conditioning. This changes only the common scale, not relative weights or the multiplicative ratio.

---

4. Common target WCS and exact detector-pixel integration

4.1 Target grid

UPDC constructs a fresh, linear TAN target WCS using:

- CRVAL / CRPIX;
- a first-order CD/PC-equivalent matrix;
- no SIP or other high-order distortion embedded in the target WCS.

The target pixel scale is derived from the first exposure's linear pixel-scale matrix and divided by `upsample`. Exposure boundaries are mapped through their own WCS/GWCS to determine a target extent that contains all input coverage.

4.2 Detector pixels are not point samples

Every detector pixel is represented by its four corners. Those corners are transformed from detector coordinates to sky, then from sky to the common target grid. The resulting quadrilateral is intersected with target pixels.

For detector pixel `p` and target pixel `i`,

$$S[p, i] = \frac{A_{overlap}(p, i)}{A_{mapped}(p)}.$$

The forward detector prediction is therefore an exact polygon-area average over the straight-sided mapped detector quadrilateral, not a center-point interpolation.

At an image boundary the overlap still divides by the full mapped detector-pixel area, so truncated edge coverage is not artificially renormalized.

4.3 LinkMap

All non-zero overlap coefficients for one exposure are precomputed into a sparse LinkMap and reused through all iterations. LinkMaps are a major performance and memory data structure in UPDC.

---

5. Detector-frame PSFs and full WCS/GWCS polygon transport

5.1 Validated JWST HybPSF workflow

The validated JWST workflow uses a detector-frame, oversampled PSF. The PSF itself may be produced by an external empirical, PCA, WebbPSF, or hybrid pipeline; **UPDC consumes the final 2-D PSF image and does not generate the PSF model**.

The frozen v17 solver uses one spatially invariant PSF per exposure. Different exposures may use different PSFs, but spatial PSF variation within a single exposure is not represented by the frozen operator.

5.2 Detector coordinates of PSF pixels

For PSF width `PW`, height `PH`, oversampling factor `s`, and detector anchor (u_0, v_0), the PSF pixel boundaries are

$$u_j = u_0 + \frac{j - PW/2}{s}, \qquad j = 0 \ldots PW,$$

$$v_i = v_0 + \frac{i - PH/2}{s}, \qquad i = 0 \ldots PH.$$

For a 2x oversampled PSF, each PSF sample corresponds to a `0.5 x 0.5` native detector-pixel element.

5.3 Full geometric transport

For every exposure:

1. every PSF-sample corner is transformed through the exposure's full detector-to-sky WCS/GWCS;
2. sky coordinates are transformed into the common linear TAN target grid;
3. the mapped detector-anchor location is subtracted from the PSF corners.

Subtracting the mapped anchor removes pointing/dither translation from the convolution kernel while retaining:

- rotation;
- scale variation;
- shear;
- high-order detector distortion as expressed over the PSF support.

Exposure 1 is transformed as well. There is no identity exception.

5.4 Flux-conserving polygon push-forward

After mapping, an input PSF sample is a quadrilateral on the target plane. Its flux is distributed by exact overlap fractions:

$$f_{out,j} \mathrel{+}= f_{in} \frac{A_{overlap,j}}{A_{quad}}.$$

No Lanczos, bilinear, or spline interpolation kernel is used for this PSF transport. This avoids interpolation-induced negative PSF lobes and preserves the geometric interpretation of the detector-frame PSF.

5.5 PSF crop

An optional centered odd-square crop is applied **after** geometric transport. In the validated JWST configuration, `psf_crop_size = 1431` typically retains about 99.98% of mapped PSF flux. The solver normalizes the retained PSF before generating its FFT spectrum.

The crop is a support/performance trade-off and does not replace the preceding distortion-aware transport.

---

6. Numerical precision policy

UPDC deliberately uses mixed precision.

6.1 float64

Double precision is reserved for topology-sensitive geometry:

- WCS/GWCS coordinates;
- polygon clipping tests;
- segment/boundary intersections;
- mapped vertices and related geometric decisions.

6.2 float32

Single precision is used for the numerical reconstruction path:

- local polygon area calculations and overlap fractions;
- SCI and WHT arrays;
- PSFs and PSF normalization;
- FFTW3f transforms;
- forward means;
- FreezeMu ratio handling;
- detector scatter;
- sensitivity denominator;
- multiplicative updates;

- target reconstruction.

To reduce cancellation from large absolute coordinates, polygon areas are evaluated after translating vertices into a local coordinate frame before conversion to float32. Long PSF sums use compensated single-precision summation.

Different FFT padding choices, FFTW plans, and threaded reduction orders can produce small float32 differences. Cross-platform validation should compare scientific tolerances rather than require bitwise identity.

---

7. Initial estimate and FreezeMu

The initial target estimate is a weighted drizzle-like combination of the input exposures:

- raw SCI is scattered through the same geometric LinkMap;
- the scalar background is scattered with the same coverage weights;
- the estimate is formed as data/coverage minus background/coverage;
- no positivity clipping is applied.

The reconstruction is therefore signed from iteration zero.

FreezeMu is a persistent detector-pixel state. If `raw_mu <= 0` occurs for a usable pixel at any iteration, that pixel remains frozen for the rest of the run and its ratio stays at 1.

Diagnostics include:

- `n_new_frozen`
- `n_total_frozen`
- `min_raw_mu`

These values are intended for numerical monitoring and regression testing.

---

8. Main computational optimizations

The frozen v17 optimizations preserve the mathematical operator.

8.1 FFT convolution

P_k and P_k^T are implemented with FFTW3f on a padded target grid.

8.2 Only `2N + 2` FFT transforms per iteration

For `N` exposures, one iteration performs:

1. one forward FFT of the reconstruction;
2. `N` exposure-specific inverse FFTs for PSF convolution;
3. `N` forward FFTs of detector scatter images;
4. one combined adjoint inverse FFT after summing the exposure adjoint spectra.

Therefore

$$N_{FFT} = 2N + 2.$$

For 9 exposures, the iterative loop uses 20 transforms per iteration. Nineteen iterations therefore use 380 transforms, excluding setup/autotune/precomputation transforms.

8.3 Fused forward + FreezeMu + scatter detector loop

For each detector pixel, one LinkMap stencil traversal performs:

```
prediction -> raw_mu -> FreezeMu -> ratio -> W*ratio -> scatter
```

This removes a separate detector-state pass and improves cache locality.

8.4 Frequency-domain adjoint reduction

Linearity gives

$$ \sum_k P_k^T s_k

F^{-1}\left[\sum_k P_k^* F(s_k)\right]. $$

UPDC therefore accumulates exposure adjoint spectra in frequency space and performs only one final inverse FFT. The sensitivity denominator uses the same reduction principle.

8.5 Persistent exposure worker pool

Exposure workers are created once and reused across the iterative solve. The solver avoids repeated `pthread_create` / `pthread_join` phases inside every iteration.

8.6 Independent parallelism controls

- `exposure_workers`: parallel exposures;
- `fft_workers`: FFTW threads within one transform;
- `preprocess_workers`: Python/C preprocessing such as WCS mapping, PSF transport, LinkMap construction, initial drizzle, and sensitivity preparation.

The fastest combination depends on exposure count, NUMA topology, cache hierarchy, memory bandwidth, FFT size, and scheduler allocation. Larger thread counts are not automatically faster.

8.7 Adaptive LinkMap v2

Each exposure independently selects its sparse target-index encoding.

DELTA16 stores a `uint32 base_target` for each detector pixel and a `uint16 delta + float32 frac` for each link when all relative target indices fit safely in 16 bits.

ABS32 stores a full `uint32 target + float32 frac` when DELTA16 is not safe.

This is capability detection, not a JWST-specific assumption. `regular-3x3` statistics are diagnostic only and do not alter the numerical path.

8.8 FFT-shape autotuning and FFTW wisdom

UPDC benchmarks a small set of valid padded FFT sizes and caches the fastest result for the relevant host/FFTW/worker/minimum-shape configuration. FFTW wisdom is also persisted.

8.9 Persistent preprocessing cache

Content-addressed dataset caches can reuse:

- normalized SCI/WHT working planes;
- target WCS metadata;
- mapped PSFs;
- detector polygon vertices;
- LinkMaps;
- PSF spectra;
- initial drizzle estimate;
- sensitivity denominator;
- compiled C binary associated with its toolchain signature.

Changing iteration count, save cadence, or worker topology should not force geometric recomputation when the scientific inputs are unchanged.

8.10 Per-run temporary staging

Raw iteration buffers and diagnostic CSV files are kept in a per-run temporary directory rather than in the persistent preprocessing cache. This prevents stale iteration files, cache growth, and collisions between concurrent runs.

9. Cache lifecycle and reset controls

Recommended configuration:

```toml
[cache]
enabled = true
reset = "none"          # none | current | all
prune_after_days = 30   # 0 disables automatic pruning
```

9.1 `reset = "none"`

Normal production mode. Reuse compatible caches whenever possible.

9.2 `reset = "current"`

At startup, delete the preprocessing cache for the current dataset key and rebuild it from scratch. Typical uses include:

- measuring cold preprocessing performance;
- recovering from a suspected corrupted dataset cache;
- intentionally rebuilding LinkMaps, PSF spectra, or the denominator.

Return the setting to `none` after the intended rebuild; reset is evaluated on every run.

9.3 `reset = "all"`

Remove all dataset caches under the configured UPDC cache root whose directory names match the expected dataset-key pattern. Unrelated paths and global FFTW wisdom are preserved.

9.4 `prune_after_days`

When greater than zero, stale dataset caches are removed the next time UPDC starts. UPDC does not run a background daemon.

9.5 Memory release

At process exit, ordinary process memory, NumPy buffers, FFTW workspaces, and mmap mappings are released automatically. Linux filesystem page cache can remain visible as `buff/cache`, but it is reclaimable by the kernel under memory pressure.

Persistent disk cache under `~/.cache/updc-jwst/data/...` remains until reset/prune/manual deletion.

---

10. TOML configuration and recommended execution

Minimal workflow:

```
updc init updc.toml
# edit paths and scientific settings
updc validate updc.toml
updc updc.toml
```

Equivalent:

```
python3 -m updc updc.toml
```

Temporary command-line override:

```
updc updc.toml --set solver.iterations=39
updc updc.toml --set performance.fft_workers=4
updc updc.toml --set 'cache.reset="current"'
```

Schema v1 is strict: unknown sections and keys are rejected rather than silently ignored.

10.1 JWST example

```toml
[updc]
schema_version = 1

[instrument]
adapter = "jwst"

[input]
input_dir = "./JWST/NIRCam/level2"
input_glob = "*_cal*.fits"
pairing = "index"
wcs_mode = "gwcs"
weight_mode = "rnoise-sky"
bad_dq_mask = 531
poisson_pedestal_sigma = 0.0

[psf]
psf_dir = "./JWST/PSF"
psf_glob = "*.fits"
psf_orientation = "stored-transposed"
psf_frame = "detector"
psf_oversample = 2
psf_crop_size = 1431
write_transformed_psf = false

[solver]
iterations = 19
upsample = 2
save_every = 5

[performance]
workers = 1
fft_workers = 2
exposure_workers = 2
preprocess_workers = 4
fft_plan = "measure"
fft_autotune = true
retune_fft = false

[cache]
enabled = true
reset = "none"
prune_after_days = 30

[output]
output_dir = "./output"
keep_work = false

[build]
cc = "cc"
```

For generic FITS, start from `examples/generic_fits.toml` rather than changing only the adapter name in the JWST template, because WCS mode, PSF orientation, crop, DQ semantics, and pairing assumptions may differ.

---

11. Installation

11.1 Python package

Python `>=3.10` is supported.

```
python -m pip install updc_reconstruction-1.1.0-py3-none-any.whl
```

For JWST/GWCS support:

```
python -m pip install 'updc-reconstruction[jwst]'
```

11.2 FFTW3f requirement

UPDC requires single-precision FFTW and its threaded wrapper:

- `fftw3.h`
- `libfftw3f`
- `libfftw3f_threads`

Ubuntu / Debian:

```
sudo apt-get install libfftw3-dev pkg-config cmake build-essential
```

macOS / Homebrew:

```
brew install fftw pkg-config cmake
```

Conda / micromamba:

```
conda install -c conda-forge fftw pkg-config
```

Then run:

```
updc doctor
```

The doctor command checks frozen-core integrity, compiler availability, C11 atomics, native CPU flags, FFTW discovery, and common HPC header-shadowing problems.

---

12. Linux and HPC deployment

12.1 GCC on AMD EPYC

The validated AMD EPYC 7763 host is Zen 3. GCC 13.3 reports

```
-march=native -> znver3
```

and is an appropriate production compiler for that system. An Intel compiler is not required merely because Intel compiler modules are present on the cluster.

12.2 C11 `stdatomic.h` header shadowing

Some HPC module environments incorrectly inject C++ include paths into the C include search path, for example:

```
C_INCLUDE_PATH=.../include/c++/13:...
```

A pure-C include such as

```
#include <stdatomic.h>
```

can then resolve to a libstdc++ compatibility header instead of GCC's C11 header, producing errors such as:

```
unknown type name atomic_uint_fast64_t
memory_order_relaxed undeclared
```

UPDC 1.1.0 filters `include/c++/...` paths from the C build environment and reports this condition through `updc doctor`.

Manual diagnostics:

```
echo '#include <stdatomic.h>' | gcc -std=c11 -H -E -x c - >/dev/null
gcc -print-file-name=include/stdatomic.h
```

If needed:

```
unset C_INCLUDE_PATH
unset CPLUS_INCLUDE_PATH
```

12.3 Native CPU flag probing

The build helper tests the native CPU flag with a tiny independent compile probe before building the full core. An unrelated source or linker error is therefore not mislabeled as an `-march=native` failure.

12.4 Toolchain-aware compiled-binary cache

The C binary cache signature includes:

- frozen source hashes;
- compiler executable path;
- compiler version;
- compile flags;
- FFTW discovery/version/link information;
- host OS and architecture.

Changing GCC/Clang, FFTW, or relevant flags therefore forces an appropriate rebuild.

---

13. Instrument adapters and generalization

Software layers:

```
native telescope products
        -> instrument adapter
        -> normalized exposure contract + PSFs
        -> Python WCS / geometry / cache orchestration
        -> frozen v17 C numerical core
        -> FITS outputs
```

Built-in adapters:

- `jwst`
- `fits`

Third-party packages can register adapters through the Python entry-point group:

```
[project.entry-points."updc.instrument_adapters"]
my-scope = "mypackage.adapter:MyAdapter"
```

Adapters may change ingestion and normalization but must not silently change the frozen numerical operator. If another telescope requires a genuinely different forward model, that capability should be introduced as a new numerical-core design with its own validation and freeze.

---

14. Choosing parallelism

Three controls are independent:

```
[performance]
fft_workers = 2
exposure_workers = 2
preprocess_workers = 4
```

General rules:

- `exposure_workers > number_of_exposures` does not add exposure parallelism;

- `fft_workers * exposure_workers` is a possible nested FFT concurrency ceiling, not a guarantee that all those cores remain saturated;
- large 2-D FFTs can become memory-bandwidth- or NUMA-limited;
- optimal settings must be benchmarked on the actual machine and scheduler allocation.

14.1 AMD EPYC 7763 reference acceptance

Validated environment:

- CPU: AMD EPYC 7763 (Zen 3)
- 64-core server, 256 GiB RAM
- GCC 13.3 (`-march=native -> znver3`)
- FFTW3f 3.3.5
- 9 exposures, each 2048 x 2048
- target: 4565 x 4530
- PSF: 1431 x 1431
- 19 iterations
- FFT autotune: 6000 x 6000
- `fft_workers = 9`
- `exposure_workers = 9`
- `preprocess_workers = 9`

Warm-cache acceptance results:

- solver wall: **21.207 s**
- total wall: **23.196 s**
- peak RSS: **9.189 GiB**
- iterative FFTs: 190 forward + 190 inverse = **380 transforms**
- nine DELTA16 LinkMaps: approximately **2.190 GiB** mapped

These measurements are regression references, not performance guarantees for other machines or datasets.

For scheduler jobs strictly limited to 64 CPUs, it can also be useful to compare `fft_workers=7, exposure_workers=9` against the 9x9 setting.

---

15. Cache ABI

Current explicit cache ABIs:

Cache	ABI	Notes
Python preprocessing	1	Marker `updc-v16-preprocess-1`, name retained for compatibility with validated caches
LinkMap	2	Magic `ULM2`, adaptive `ABS32` / `DELTA16`
FFT autotune	1	`fft_autotune_v1_*`

Rules:

1. Readers must check magic/version/shape/count/encoding.
2. An incompatible persistent format change must increment its ABI.
3. An unknown ABI is a cache miss and requires rebuild.
4. Cache files are performance artifacts, not scientific products.
5. Cache hit and cache miss must not change the mathematical operator.

---

16. Outputs and diagnostics

Typical iteration products are named like:

```
updc_<filter>_<detector>_iter05_i2d.fits
updc_<filter>_<detector>_iter10_i2d.fits
...
```

`save_every = 0` writes only the final iteration.

Each iteration timing line can include:

- `estimateFFT`
- `fusedWall`
- `forwardIFFTcrit`
- `detectorCrit`
- `scatterFFTcrit`
- `adjointReduceIFFT`
- `update`
- `output`

These diagnostics help distinguish FFT, sparse detector-loop, memory-bandwidth, and output bottlenecks.

Output FITS products include reconstructed `SCI`, pre-PSF coverage/weight `WHT`, scalar-background diagnostics, PSF-geometry diagnostics, and FreezeMu metadata. `WHT` should not be interpreted as a full post-deconvolution inverse covariance map.

---

17. Testing and scientific regression

17.1 Python/package tests

```
python -m pip install '.[test]'
pytest -q
```

Coverage includes:

- frozen-core hashes;
- cache ABI;
- strict TOML schema;
- adapter registry;
- CLI;
- cache reset/pruning;
- engine handoff;
- PSF geometry;
- build portability and C++ header shadowing;
- C11 atomics probes and toolchain signatures;
- release metadata.

17.2 C core

Recommended:

```
cmake -S updc/c_core -B build -DUPDC_NATIVE_ARCH=ON
cmake --build build --config Release
ctest --test-dir build --output-on-failure
```

17.3 Scientific output comparison

Do not use bitwise identity as the only cross-platform criterion. Compare:

- maximum absolute difference;
- RMS difference;
- total reconstruction sum/flux diagnostics;
- negative-pixel count;
- WHT;
- FreezeMu counts and minimum raw mu;
- output WCS/header metadata.

Utility:

```
python tools/compare_fits.py reference.fits candidate.fits
```

---

18. GitHub release workflow

Before release:

```
python -m pytest -q
updc doctor
python -m build
python -m twine check dist/*
```

Also verify the frozen C hashes and update release metadata and checksums.

Recommended release assets:

- source ZIP and/or tar.gz;
- Python wheel;
- `UPDC_COMPLETE_MANUAL_EN.pdf`;
- `UPDC_ALGORITHM_MANUAL_EN.pdf`;
- SHA-256 checksum list.

The repository should contain source, tests, CI, configuration examples, and documentation; generated distribution binaries are better attached as GitHub Release assets than committed to the source tree.

---

19. Known boundaries and capabilities that should not be overstated

1. **UPDC is not a PSF-generation package.** It consumes PSFs produced elsewhere.
2. **The frozen v17 core is not a spatially varying PSF solver within one exposure.** Different exposures may use different kernels, but a single exposure uses one kernel.
3. **JWST has end-to-end real-data acceptance. Other telescopes require adapter work and independent scientific validation.** A generic software architecture is not the same as instrument calibration.
4. **Cache files are performance artifacts, not science products.** They must be reproducibly rebuildable.
5. **Different FFTW plans and thread-reduction orders can produce small float32 differences.** Bitwise equality is not a universal cross-platform scientific requirement.
6. **More threads do not guarantee lower wall time.** NUMA placement, memory bandwidth, exposure count, FFT size, and scheduler limits matter.
7. **The target WCS is linear TAN.** High-order detector distortion is handled through source-to-target mapping, not duplicated into the target WCS.
8. **Detector geometry uses mapped corner quadrilaterals.** It is exact polygon integration for those straight-sided mapped pixels, not continuous analytic integration of curvature inside a detector pixel.

---

20. Recommended production principles

- Choose scientific parameters - PSF, WCS mode, weights, DQ mask, pedestal policy, and iteration count - from the data and science goal.
- Choose performance parameters by benchmarking the actual machine.
- Use `cache.reset = "none"` for routine production.
- Set `reset = "current"` only when an explicit rebuild is intended, then restore `none`.
- `prune_after_days = 30` is a reasonable shared-server policy; use `0` for fully manual cache retention.
- On Linux/HPC systems, run `updc doctor` before production to verify compiler, C11 atomics, native CPU flags, and FFTW.
- Do not modify the frozen v17 C files in a 1.1.x release. A numerical change requires a new core version and validation.

---

21. Version information

UPDC package 1.1.0

- numerical core: frozen v17;

- stable TOML schema: v1;
- LinkMap ABI: v2;
- preprocessing cache ABI: v1;
- FFT-autotune cache ABI: v1;
- cache reset/prune: supported;
- Linux/HPC build sanitation and probes: supported;
- instrument adapter API: supported;
- primary CLI: `updc config.toml` or `python -m updc config.toml`.

---

22. MIT License and scientific publication citation policy

UPDC 1.1.0 is distributed under the standard MIT License in the repository-root `LICENSE`. The MIT License allows use, copying, modification, merging, publication, distribution, sublicensing, and/or sale of copies subject to preservation of the copyright and permission notice, and it provides the software "AS IS" without warranty.

For journal articles, conference proceedings, theses/dissertations, technical reports, data-release papers, or other publications that use UPDC code, its algorithmic implementation, or scientific results produced with UPDC, the UPDC project requires citation of **both** publications:

1. Wang, L., Li, G., & Kang, X. 2022, *Towards super-resolution via iterative multi-exposure coaddition*, **MNRAS, 517, 787**, doi: `10.1093/mnras/stac2664`.
2. Wang, L., Shan, H., Nie, L., et al. 2025, *UPdec-Webb: A Data Set for Coaddition of JWST NIRCam Images*, **ApJS, 276, 36**, doi: `10.3847/1538-4365/ad9566`.

Machine-readable citation metadata are in `CITATION.cff`, and BibTeX entries are in `CITATION.md`.

This publication-citation policy is a scientific-attribution policy of the UPDC project. It is separate from, and does not modify, the standard MIT software license.